

Dining Philosophers :

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5 // number of philosophers

pthread_mutex_t forks[N];

void* philosopher(void* num) {
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;

    while (1) {
        printf("Philosopher %d is thinking 🧠\n", id);
        sleep(1);

        // To prevent deadlock:
        // Even philosophers pick up right fork first, odd
        // pick up left first.
        if (id % 2 == 0) {
            pthread_mutex_lock(&forks[right]);
            pthread_mutex_lock(&forks[left]);
        } else {
            pthread_mutex_lock(&forks[left]);
            pthread_mutex_lock(&forks[right]);
        }

        printf("Philosopher %d is eating 🍽️\n", id);
        sleep(1);

        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);

        printf("Philosopher %d finished eating 🍴\n", id);
```

```

    }
}

int main() {
    pthread_t threads[N];
    int ids[N];

    for (int i = 0; i < N; i++)
        pthread_mutex_init(&forks[i], NULL);

    for (int i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, philosopher,
&ids[i]);
    }

    for (int i = 0; i < N; i++)
        pthread_join(threads[i], NULL);

    for (int i = 0; i < N; i++)
        pthread_mutex_destroy(&forks[i]);

    return 0;
}

```

Producer Consumer

```

#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<semaphore.h>
#include<unistd.h>

#define BUFFER_SIZE 5
#define NUM_ITEMS 10 // kitte produce karega

int buffer[BUFFER_SIZE];
int in = 0 , out = 0 ;

```

```

sem_t empty ;
sem_t full ;
pthread_mutex_t mutex ;

void * producer ( void * a ){
    for ( int i = 1 ; i <= NUM_ITEMS ; i++){
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i ;
        printf("Produced %d\n" , i );
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}

void * consumer ( void * a ){

    for (int i = 1 ; i <= NUM_ITEMS ; i++){
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed : %d\n" , item);
        out = (out+1) %BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(1);
    }
    return NULL;
}

int main(){
    pthread_t prod , cons ;

    sem_init(&empty,0,BUFFER_SIZE);

```

```

sem_init(&full,0,0);
pthread_mutex_init(&mutex,NULL);

pthread_create(&prod,NULL,producer,NULL);
pthread_create(&con[],NULL,consumer,NULL);

pthread_join(prod, NULL);
pthread_join(cons, NULL);

sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

return 0 ;
}

```

Reader writer

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t wrt;           // Controls writer access
pthread_mutex_t mutex; // Protects readcount
int readcount = 0;    // Number of readers currently
reading
int data = 0;         // Shared data

void* reader(void* arg) {
    int id = *(int*)arg;

    while (1) {
        pthread_mutex_lock(&mutex);
        readcount++;
        if (readcount == 1)
            sem_wait(&wrt); // first reader locks writer
        pthread_mutex_unlock(&mutex);
    }
}

```

```

        printf("Reader %d is reading data = %d 📖\n", id,
data);
        sleep(1);

        pthread_mutex_lock(&mutex);
        readcount--;
        if (readcount == 0)
            sem_post(&wrt); // last reader unlocks writer
        pthread_mutex_unlock(&mutex);

        sleep(1);
    }
}

void* writer(void* arg) {
    int id = *(int*)arg;

    while (1) {
        sem_wait(&wrt); // wait for access
        data++;
        printf("Writer %d modified data to %d 🖋️\n", id,
data);
        sleep(1);
        sem_post(&wrt); // release access
        sleep(1);
    }
}

int main() {
    pthread_t rtid[5], wtid[2];
    int rid[5], wid[2];

    sem_init(&wrt, 0, 1);
    pthread_mutex_init(&mutex, NULL);

    for (int i = 0; i < 5; i++) {
        rid[i] = i + 1;
        pthread_create(&rtid[i], NULL, reader, &rid[i]);
    }
}

```

```

    for (int i = 0; i < 2; i++) {
        wid[i] = i + 1;
        pthread_create(&wtid[i], NULL, writer, &wid[i]);
    }

    for (int i = 0; i < 5; i++)
        pthread_join(rtid[i], NULL);
    for (int i = 0; i < 2; i++)
        pthread_join(wtid[i], NULL);

    sem_destroy(&wrt);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

Q1. Message Passing with Pipes

Parent sends an array → Child sums → Sends result back.

```

#include <stdio.h>
#include <unistd.h>

int main() {
    int fd1[2], fd2[2];
    pipe(fd1); // parent to child
    pipe(fd2); // child to parent

    if (fork() == 0) { // Child
        close(fd1[1]); // close write end of first pipe
        close(fd2[0]); // close read end of second pipe

        int arr[5];
        read(fd1[0], arr, sizeof(arr)); // read array
        int sum = 0;
        for (int i = 0; i < 5; i++) sum += arr[i];
        write(fd2[1], &sum, sizeof(sum)); // send back sum
    }
}

```

```

    }
    else { // Parent
        close(fd1[0]);
        close(fd2[1]);

        int arr[5] = {1, 2, 3, 4, 5}, sum;
        write(fd1[1], arr, sizeof(arr));
        read(fd2[0], &sum, sizeof(sum));
        printf("Sum received from child: %d\n", sum);
    }
    return 0;
}

```

🧠 **Concept:** Two unnamed pipes → one for each direction (parent→child, child→parent).

Q2. Shared Memory Communication

Process 1 writes a string → Process 2 reads and converts to uppercase.

```

#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int shmid = shmget(IPC_PRIVATE, 1024, 0666 |
IPC_CREAT);
    char *str = (char*) shmat(shmid, NULL, 0);

    if (fork() == 0) { // Child (Process 2)
        sleep(1); // wait for parent
        for (int i = 0; str[i]; i++)

```

```

        str[i] = toupper(str[i]);
        printf("Child: Converted to uppercase: %s\n", str);
        shmdt(str);
    }
    else { // Parent (Process 1)
        strcpy(str, "hello shared memory");
        printf("Parent: Wrote string: %s\n", str);
        wait(NULL);
        shmdt(str);
        shmctl(shmid, IPC_RMID, NULL);
    }
    return 0;
}

```

 **Concept:** Shared memory (`shmget` , `shmat`) creates a region accessible to both parent and child.

Q3. Producer–Consumer Problem using Semaphores

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5
int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    int item = 0;
    while (1) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

```

```

        buffer[in] = ++item;
        printf("Produced: %d\n", item);
        in = (in + 1) % SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
}

void* consumer(void* arg) {
    int item;
    while (1) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(1);
    },
}

int main() {
    pthread_t p, c;
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}

```

 **Concept:** Semaphores control buffer availability; mutex prevents concurrent access to shared indices.



Q4. Race Condition Demonstration

(a) Without Synchronization

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void* increment(void* arg) {
    for (int i = 0; i < 100000; i++)
        counter++;
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Final counter (no mutex): %d\n", counter);
    return 0;
}
```

(b) With Mutex (Corrected Version)

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        counter++;
    }
}
```

```

        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_mutex_init(&lock, NULL);
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Final counter (with mutex): %d\n", counter);
    pthread_mutex_destroy(&lock);
    return 0;
}

```

Named semaphores

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS_PER_PROD 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0, count = 0;

sem_t *empty, *full;
sem_t *mutex;

void* producer(void* arg) {
    int id = *(int*)arg;
    for (int i = 1; i <= ITEMS_PER_PROD; i++) {

```

```

        int item = id * 100 + i;
        sem_wait(empty);
        sem_wait(mutex);
        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        count++;
        printf("P%d produced %d\n", id, item);
        fflush(stdout);
        sem_post(mutex);
        sem_post(full);
        usleep(200000);
    }
    return NULL;
}

void* consumer(void* arg) {
    int id = *(int*)arg;
    while (1) {
        sem_wait(full);
        sem_wait(mutex);
        if (count == 0) { sem_post(mutex); sem_post(full);
break; }
        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;
        printf("                C%d consumed %d\n", id,
item); fflush(stdout);
        sem_post(mutex);
        sem_post(empty);
        usleep(300000);
    }
    return NULL;
}

int main() {
    empty = sem_open("/empty_sem", O_CREAT, 0644,
BUFFER_SIZE);
    full  = sem_open("/full_sem",   O_CREAT, 0644, 0);
    mutex = sem_open("/mutex_sem",  O_CREAT, 0644, 1);

```

```

    int np = 3, nc = 2;
    pthread_t prod[np], cons[nc];
    int ids[np + nc];
    for (int i = 0; i < np; i++) ids[i] = i+1;
    for (int i = 0; i < nc; i++) ids[np+i] = i+1;

    for (int i = 0; i < np; i++) pthread_create(&prod[i],
NULL, producer, &ids[i]);
    for (int i = 0; i < nc; i++) pthread_create(&cons[i],
NULL, consumer, &ids[np+i]);

    for (int i = 0; i < np; i++) pthread_join(prod[i],
NULL);
    for (int i = 0; i < nc; i++) pthread_join(cons[i],
NULL);

    sem_close(empty); sem_close(full); sem_close(mutex);
    sem_unlink("/empty_sem"); sem_unlink("/full_sem");
    sem_unlink("/mutex_sem");

    printf("Q2 Done\n");
    return 0;
}

```